Artificial Intelligence

Viva Preparation Guide

GTU Subject Code: 3170716

1. What is AI

2. Neural Network

3. What is Neural

4. Bias

5. Threshold

6. Activation Function

7. Mapping

8. Prolog Installation Steps

9. Applications of AI

10. Fuzzy Logic and Applications

11. Genetic Algorithm and Process

12. Neural Network Parameters

13. Bayes' Theorem / Bayes' Weight

14. BFS, DFS and A* Algorithm

15. Heuristic Search Algorithms

1. What is Artificial Intelligence (AI)?

Artificial Intelligence is the branch of computer science that aims to build machines and software that can perform tasks which normally need human intelligence, such as learning, reasoning, problem solving, perception, understanding language and decision making. The term was coined by **John McCarthy** in 1956 (Dartmouth Conference).

Four approaches to AI (Russell and Norvig)

Approach	Meaning
Thinking humanly	Cognitive modelling: machines that think like humans
Acting humanly	Turing Test approach: behaviour indistinguishable from a human
Thinking rationally	Laws of thought: logic-based reasoning
Acting rationally	Rational agent approach: act to achieve the best expected outcome

Types of AI

- **Narrow (Weak) AI:** built for one task (chess, spam filter, Siri).
- **General (Strong) AI:** human-level ability across any task (not yet achieved).
- **Super AI:** surpasses human intelligence (hypothetical).

Main branches

- Machine Learning, Neural Networks and Deep Learning
- Natural Language Processing (NLP)
- Expert Systems
- Robotics
- Computer Vision
- Fuzzy Logic and Genetic Algorithms

> **Viva one-liner:** AI is the science of making machines act intelligently, i.e. learn, reason and take decisions like humans.

2. What is a Neural Network?

An **Artificial Neural Network (ANN)** is a computing model inspired by the human brain. It is made of many simple processing units called **neurons (nodes)**, connected by links that carry **weights**. The network learns by adjusting these weights from training data.

Structure

- **Input layer:** receives raw data (features).
- **Hidden layer(s):** perform intermediate computation and feature extraction.
- **Output layer:** produces the final result (class or value).

Working (step by step)

1. Inputs x1, x2, ..., xn are multiplied by their weights w1, w2, ..., wn.
2. The weighted sum is calculated and bias is added.

3. The sum is passed through an activation function.
4. The output is sent to the next layer or given as the final answer.
5. Error is calculated and weights are updated (learning, e.g. backpropagation).

```
x1 --w1--\
x2 --w2---> [ SUM(wi*xi) + b ] --> [ Activation f ] --> y
x3 --w3--/
```

Types

- Single-layer Perceptron
- Multi-layer Perceptron (MLP) with backpropagation
- Feed-forward network
- Recurrent Neural Network (RNN)
- Convolutional Neural Network (CNN)
- Self-organising map (Kohonen)

Learning types

- **Supervised:** trained with labelled data.
- **Unsupervised:** finds patterns in unlabelled data.
- **Reinforcement:** learns by reward and penalty.

> **Viva one-liner:** A neural network is a layered network of weighted, connected neurons that learns patterns by adjusting its weights.

3. What is Neural?

Neural is an adjective that means *relating to a nerve, a neuron or the nervous system*. In Artificial Intelligence, the word 'neural' refers to computing methods that are **inspired by the structure and working of the human brain**, which is a huge network of interconnected nerve cells (neurons). That is why we say **neural network**, **neural computing** and **neural processing**.

Biological basis of 'neural'

The human brain has about 86 billion neurons. Each biological neuron has:

- **Dendrites:** receive signals from other neurons.
- **Cell body (soma):** processes the incoming signals.
- **Axon:** carries the output signal away.
- **Synapse:** the junction between two neurons whose strength can change (this is how the brain learns).

Neural computing in AI

Neural computing is an approach in which many simple processing units work in parallel and learn from examples instead of being explicitly programmed with rules. The artificial version of the biological structure is:

Biological (neural) system	Artificial neural system
Neuron	Node / processing unit

Biological (neural) system	Artificial neural system
Dendrites	Inputs
Synapse strength	Weights
Cell body (soma)	Summation + activation function
Axon	Output

Features of neural systems

- **Learning ability:** improve performance from data and experience.
- **Parallel processing:** many units compute at the same time.
- **Fault tolerance:** the system still works if some units fail.
- **Generalisation:** gives sensible output for unseen inputs.
- **Adaptability and non-linearity:** can model complex relationships.

Mathematical model of one artificial neural unit:

```
net = (w1*x1 + w2*x2 + ... + wn*xn) + b
y   = f(net)
```

Related terms: neural network, neural computation, neuro-fuzzy systems (neural + fuzzy logic), neuroscience (study of the nervous system).

> **Viva one-liner:** 'Neural' means brain-like (relating to neurons); in AI it describes systems modelled on the brain's network of neurons.

4. What is Bias?

Bias (b) is an extra constant parameter added to the weighted sum of a neuron. It allows the activation function to be **shifted left or right**, so the neuron can produce a non-zero output even when all inputs are zero. It is like the intercept *c* in the line equation y = mx + c.

```
net = SUM(wi * xi) + b
```

- It is a learnable parameter, just like weights, and is updated during training.
- Without bias, the decision boundary must pass through the origin, which limits the model.
- Bias can be viewed as a weight attached to a fixed input of +1.
- Bias in the sense of 'data bias' (unfair training data) is a different concept and also exists in AI ethics.

> **Viva one-liner:** Bias shifts the activation function so the network can fit data that does not pass through the origin.

5. What is Threshold?

Threshold (theta) is the minimum value the net input must reach for a neuron to **fire** (activate). In a threshold (step) activation function:

```
y = 1   if net >= threshold
y = 0   if net <  threshold
```

- Used in McCulloch-Pitts neurons and the perceptron.

- Bias and threshold are related: **bias = - threshold**. Writing net - threshold >= 0 is the same as net + b >= 0.
- Example: AND gate with w1 = w2 = 1 and threshold = 2 fires only when both inputs are 1.

Viva one-liner: Threshold is the cut-off value of net input above which a neuron produces output 1.

6. Activation Function

An **activation function** decides the output of a neuron from its net input. It introduces **non-linearity**, which lets the network learn complex patterns. Without it, a multi-layer network would behave like a single linear model.

Common activation functions

Function	Formula	Output range	Remarks
Step / Threshold	f(x) = 1 if x >= 0, else 0	0 or 1	Perceptron; not differentiable
Linear / Identity	f(x) = x	-inf to +inf	Used in output layer for regression
Sigmoid (logistic)	f(x) = 1 / (1 + e^(-x))	0 to 1	Probability output; vanishing gradient
Tanh	f(x) = (e^x - e^-x) / (e^x + e^-x)	-1 to 1	Zero-centred; better than sigmoid
ReLU	f(x) = max(0, x)	0 to +inf	Fast, most popular in hidden layers
Leaky ReLU	f(x) = x if x > 0 else 0.01x	-inf to +inf	Avoids dead neurons
Softmax	e^xi / SUM(e^xj)	0 to 1 (sum = 1)	Multi-class output layer

Viva one-liner: The activation function adds non-linearity and decides whether and how strongly a neuron fires.

7. Mapping

Mapping means a relationship that assigns each input to an output, written as **f : X -> Y**. A neural network is basically a **function approximator** that learns the mapping from input space to output space.

- **Input-output mapping:** the network learns from examples (x, d) so that output y is close to desired output d.
- **Pattern association / mapping networks:** store pairs of patterns, e.g. Hopfield, BAM (Bidirectional Associative Memory).
- **Auto-associative mapping:** input and output are the same pattern (noise removal, recall).
- **Hetero-associative mapping:** input is mapped to a different pattern.
- **Feature mapping:** Kohonen Self-Organising Map maps high-dimensional input to a low-dimensional (2-D) grid.
- **Fuzzy mapping:** maps crisp values to membership degrees in [0, 1].

Example: mapping of an image (pixels) to the label 'cat' or 'dog'; mapping of house features to its price.

> **Viva one-liner:** Mapping is the function that connects inputs to outputs; training a neural network means learning this mapping.

8. Prolog Installation Steps

Prolog (Programming in Logic) is a declarative logic programming language widely used in AI. The most popular free implementation is **SWI-Prolog**.

On Windows

1. Open the official site **www.swi-prolog.org** and go to **Download**.
2. Download the **Stable release** installer for Windows (64-bit .exe).
3. Run the installer, accept the license agreement and click **Next**.
4. Tick the options **'Add swipl to the system PATH'** and **'Associate .pl files with SWI-Prolog'**.
5. Choose the install folder (default is fine) and click **Install**, then **Finish**.
6. Open **SWI-Prolog (swipl-win)** from the Start menu. You will see the prompt `?-`.

On Linux (Ubuntu / Debian)

```
sudo apt-add-repository ppa:swi-prolog/stable
sudo apt update
sudo apt install swi-prolog
swipl
```

On macOS

```
brew install swi-prolog
swipl
```

Running your first program

1. Open Notepad / any editor and write the facts below. Save as **hello.pl**.
2. Start SWI-Prolog and load the file with `consult('hello.pl').` or File > Consult.
3. Ask a query at the `?-` prompt and press Enter.

```
% hello.pl
parent(john, mary).
parent(mary, sam).
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).

?- grandparent(john, Who).
Who = sam.
```

Useful commands: `halt.` to exit, `listing(parent).` to view facts, `make.` to reload changed files.

> **Viva one-liner:** Prolog is a logic programming language made of facts, rules and queries; SWI-Prolog is installed from swi-prolog.org.

9. Applications of AI

Domain	Applications
Healthcare	Disease diagnosis, medical imaging analysis, drug discovery, robotic surgery
Education	Intelligent tutoring, personalised learning, automatic grading
Finance and Banking	Fraud detection, credit scoring, algorithmic trading, chatbots
Agriculture	Crop monitoring, disease detection, yield prediction, smart irrigation
Transport	Self-driving cars, traffic management, route planning (Google Maps)
E-commerce and Entertainment	Recommendation systems (Amazon, Netflix, YouTube)
Natural Language Processing	Machine translation, speech recognition, virtual assistants, ChatGPT-style chatbots
Computer Vision	Face recognition, object detection, OCR, surveillance
Robotics and Manufacturing	Industrial robots, quality inspection, predictive maintenance
Gaming	Game-playing agents (Chess, Go), NPC behaviour
Expert Systems	MYCIN (medical), DENDRAL (chemistry), XCON (computer configuration)
Cyber Security	Intrusion and malware detection, spam filtering

Viva one-liner: AI is applied in healthcare, finance, transport, education, robotics, NLP and many more fields.

10. Fuzzy Logic and Its Applications

Fuzzy logic, introduced by **Lotfi A. Zadeh in 1965**, is a form of many-valued logic in which truth values range continuously between **0 and 1** instead of only True (1) or False (0). It handles vague, imprecise and uncertain information, e.g. 'the room is *quite warm*'.

Key concepts

- **Fuzzy set:** each element has a **membership value** mu(x) between 0 and 1.
- **Membership functions:** triangular, trapezoidal, Gaussian.
- **Linguistic variables:** e.g. Temperature = {Cold, Warm, Hot}.
- **Operations:** Union = max(muA, muB); Intersection = min(muA, muB); Complement = 1 - muA.

Fuzzy inference system (process)

1. **Fuzzification:** convert crisp input to fuzzy membership values.
2. **Rule base:** IF-THEN rules, e.g. IF temperature is HIGH THEN fan speed is FAST.
3. **Inference engine:** apply rules to the fuzzified inputs (Mamdani or Sugeno).
4. **Defuzzification:** convert the fuzzy output back to a crisp value (centroid, mean of maxima).

Crisp vs fuzzy

Crisp (classical) logic	Fuzzy logic
Truth is 0 or 1	Truth is any value from 0 to 1

Crisp (classical) logic	Fuzzy logic
Sharp boundaries	Gradual boundaries
Cannot model vagueness	Models human-like approximate reasoning

Applications

- Washing machines (load and dirt sensing), air conditioners, refrigerators, microwave ovens
- Automatic gear / ABS braking systems in cars
- Camera auto-focus and image stabilisation
- Traffic control and elevator control
- Medical diagnosis and decision support
- Industrial process and temperature control, robotics, stock market prediction

Viva one-liner: Fuzzy logic handles partial truth with degrees between 0 and 1, using fuzzification, rules and defuzzification.

11. Genetic Algorithm (Genetic Search) and Its Process

A **Genetic Algorithm (GA)** is a search and optimisation technique inspired by **natural selection and genetics** (Darwin's 'survival of the fittest'), proposed by **John Holland**. It works on a population of candidate solutions and evolves them over generations.

Terminology

Term	Meaning
Gene / Chromosome	One parameter / a full candidate solution (often a bit string)
Population	Set of chromosomes (solutions)
Fitness function	Measures how good a solution is
Generation	One iteration of the algorithm

Process (steps)

1. **Initialisation:** create a random initial population.
2. **Fitness evaluation:** compute fitness of each individual.
3. **Selection:** choose fitter parents (roulette wheel, tournament, rank selection).
4. **Crossover (recombination):** combine two parents to create offspring (single-point, two-point, uniform).
5. **Mutation:** randomly flip a few bits to maintain diversity and avoid local optima.
6. **Replacement:** form the new generation.
7. **Termination:** stop when max generations reached or a satisfactory fitness is found; otherwise repeat from step 2.

```
Initialise population
repeat
    evaluate fitness
    select parents
    crossover  -> offspring
```

```
    mutation
    form new population
until termination condition met
```

Example (crossover): Parent1 = 1101|001, Parent2 = 0110|110 gives Child1 = 1101110, Child2 = 0110001.

Applications: travelling salesman problem, scheduling, neural network training, function optimisation, game playing, machine learning feature selection.

> **Viva one-liner:** GA evolves a population of solutions using selection, crossover and mutation until the fittest solution is found.

12. Neural Network and Its Parameters

Parameters of a neural network are of two kinds: those **learned** from data and those **set by the designer** (hyperparameters).

A. Learnable parameters

- **Weights (w):** strength of connection between neurons; the main knowledge of the network.
- **Bias (b):** shifts the activation function.

B. Hyperparameters

Parameter	Description
Learning rate (alpha / eta)	Step size of weight update; too high causes divergence, too low makes learning slow
Number of layers and neurons	Defines network depth and capacity
Activation function	Adds non-linearity (sigmoid, ReLU, tanh...)
Epochs	Number of complete passes over the training data
Batch size	Number of samples processed before a weight update
Momentum	Adds a fraction of the previous update to speed up and smooth learning
Loss (cost) function	Measures error: MSE, cross-entropy
Optimizer	Gradient descent, SGD, Adam
Threshold	Firing cut-off in threshold neurons
Regularisation / dropout	Reduces overfitting

Weight update rules

```
Perceptron rule : w(new) = w(old) + eta * (d - y) * x
Gradient descent: w(new) = w(old) - eta * (dE / dw)
```

Here d is the desired output, y the actual output and E the error.

> **Viva one-liner:** Weights and biases are learned; learning rate, layers, epochs and activation function are hyperparameters chosen by us.

13. Bayes' Theorem / Bayes' Weight

*Note: 'Bayes' weight' is usually asked as Bayes' theorem and how evidence **weights** a belief (prior to posterior). Learn the theorem below; it covers the topic.*

Bayes' theorem (Thomas Bayes) gives the probability of a hypothesis H after seeing evidence E:

```
                P(E | H) * P(H)
P(H | E) = ---------------------
                      P(E)
```

Term	Meaning
P(H)	Prior probability: belief in H before evidence
P(E | H)	Likelihood: probability of evidence if H is true (acts as the weight of the evidence)
P(E)	Marginal probability (normalising factor) of the evidence
P(H | E)	Posterior probability: updated belief after evidence

Solved example

1% of people have a disease. A test detects it 90% of the time if present, and gives a false positive 5% of the time. If a person tests positive, what is the probability of actually having the disease?

```
P(D)=0.01, P(+|D)=0.90, P(+|not D)=0.05
P(+) = 0.90*0.01 + 0.05*0.99 = 0.009 + 0.0495 = 0.0585
P(D|+) = (0.90*0.01) / 0.0585 = 0.154  (about 15.4%)
```

Uses in AI

- Naive Bayes classifier (spam filtering, text classification)
- Bayesian belief networks for reasoning under uncertainty
- Medical diagnosis, speech recognition, robotics localisation

Naive Bayes assumption: all features are conditionally independent given the class.

> **Viva one-liner:** Bayes' theorem updates a prior belief into a posterior using the likelihood of the observed evidence.

14. BFS, DFS and A* Algorithm

These are **graph / tree search algorithms** used to find a path from a start node to a goal node. BFS and DFS are **uninformed (blind)** searches; A* is an **informed** search.

Breadth First Search (BFS)

- Explores the graph **level by level**: all nodes at depth d before depth d+1.
- Data structure: **Queue (FIFO)**.
- Complete: yes. Optimal: yes if all step costs are equal (finds the shallowest goal).
- Time and space: O(b^d), where b = branching factor and d = depth of the shallowest goal. Memory is its big drawback.

```
1. Put start node in the queue, mark visited
2. Remove front node; if it is goal -> stop
3. Add all unvisited neighbours to the rear of queue
4. Repeat from step 2 until queue is empty
```

Depth First Search (DFS)

- Explores as **deep as possible** along one branch, then backtracks.
- Data structure: **Stack (LIFO)** or recursion.
- Complete: no (can loop in infinite or cyclic graphs unless visited nodes are tracked). Optimal: no.
- Time: O(b^m); space: O(b*m), where m = maximum depth. Needs much less memory than BFS.

```
1. Push start node on the stack, mark visited
2. Pop top node; if it is goal -> stop
3. Push all unvisited neighbours
4. Repeat from step 2 until stack is empty
```

BFS vs DFS

Point	BFS	DFS
Data structure	Queue (FIFO)	Stack (LIFO)
Strategy	Level by level	Branch by branch (deep first)
Shortest path	Guaranteed (unit cost)	Not guaranteed
Memory	High, O(b^d)	Low, O(b*m)
Best for	Shortest path, shallow goals	Deep solutions, puzzles, topological sort

A* Algorithm

A* is a **best-first, informed search** that finds the least-cost path from start to goal. It combines the cost already paid and the estimated remaining cost:

```
f(n) = g(n) + h(n)

g(n) = actual cost from start to node n
h(n) = heuristic estimate of cost from n to goal
f(n) = estimated total cost of path through n
```

Algorithm steps

1. Create an **OPEN** list (nodes to explore) with the start node and an empty **CLOSED** list.
2. Pick the node n in OPEN with the **lowest f(n)**.
3. If n is the goal, stop and trace back the path.
4. Otherwise move n to CLOSED and generate its successors.
5. For each successor compute g, h and f. Add it to OPEN, or update it if a cheaper path is found.
6. Repeat from step 2 until the goal is found or OPEN is empty (failure).

- **Admissible heuristic:** h(n) never overestimates the true cost. Then A* is **optimal**.
- **Consistent (monotone) heuristic:** h(n) <= cost(n, n') + h(n').
- Complete: yes. Space is large since all nodes are stored.
- If h(n) = 0 A* becomes Dijkstra's algorithm; if g(n) = 0 it becomes Greedy Best-First search.

Applications: GPS and route finding, game pathfinding, robot navigation, 8-puzzle.

> **Viva one-liner:** BFS uses a queue and finds the shortest path, DFS uses a stack and saves memory, A* uses f = g + h to find the optimal path efficiently.

15. Heuristic Search Algorithms

A **heuristic** is a rule of thumb or estimate that guides the search toward the goal. A **heuristic function h(n)** estimates the cost from node n to the goal. Heuristic (informed) search uses this knowledge to reduce the search space and find solutions faster than blind search.

Example (8-puzzle): h1 = number of misplaced tiles; h2 = sum of Manhattan distances of tiles from their goal positions.

Common heuristic search algorithms

Algorithm	Evaluation function	Key idea	Optimal?
Greedy Best-First	f(n) = h(n)	Always expands the node that looks closest to goal	No
A*	f(n) = g(n) + h(n)	Balances cost so far and estimated cost left	Yes (admissible h)
Hill Climbing	Move to better neighbour	Local search; no memory of past; may get stuck	No
Simulated Annealing	Accept worse moves with probability	Escapes local maxima using a cooling schedule	No (probabilistic)
Beam Search	Keep best k nodes	Limited-width best-first	No
IDA*	Iterative deepening on f	A* with low memory	Yes
Genetic Algorithm	Fitness function	Population-based evolutionary search	No (near-optimal)

Hill climbing problems

- **Local maximum:** better than neighbours but not the best overall.
- **Plateau:** flat area where all neighbours have equal value.
- **Ridge:** a narrow peak that is hard to climb with single moves.

Heuristic properties

- **Admissible:** never overestimates the true cost.
- **Consistent:** satisfies the triangle inequality.
- **More informed heuristic** (higher h but still admissible) expands fewer nodes.

Uninformed vs informed search

Uninformed (blind)	Informed (heuristic)
No domain knowledge; BFS, DFS, UCS	Uses heuristic h(n); Greedy, A*
Explores many nodes; slower	Explores fewer nodes; faster
Guaranteed but expensive	Efficient if heuristic is good

Viva one-liner: Heuristic search uses an estimate h(n) of the distance to goal to guide the search and reduce the nodes explored.

Best of luck for your viva!